

Chapter 6: Input / Output

Input/output or I/O is the communication between an information processing system, such as a computer, and the outside world, possibly a human or another information processing system. Inputs are the signals or data received by the system and outputs are the signals or data sent from it. The term can also be used as part of an action; to "perform I/O" is to perform an input or output operation.

Besides providing abstractions like processes, address spaces and files, and OS also controls all of the computer's I/O devices.

For that, the OS should:

- issue commands to handle the I/O devices
- catch the interrupts
- handle the errors
- provide the interface between devices and rest of the system.

PRINCIPLES OF I/O HARDWARE

- Different people look at I/O hardware differently.
- Electrical engineers look in terms of chips, wires, motors, power supplies and other physical components that make up the hardware.
- Programmers look at the interface presented to the software (the commands the hardware accepts, the function it carries out and the errors that can be reported back).
- We are concerned with programming I/O devices, not designing, building or maintaining them.
- Our interest is restricted to how the hardware is programmed not, how it works inside.

I/O devices

I/O devices are the pieces of hardware used by a human (or other system) to communicate with a computer. For instance, a keyboard or computer mouse is an input device for a computer, while monitors and printers are output devices. Devices for communication between computers, such as modems and network cards, typically perform both input and output operations.

I/O devices can be roughly divided into:

1. Block devices

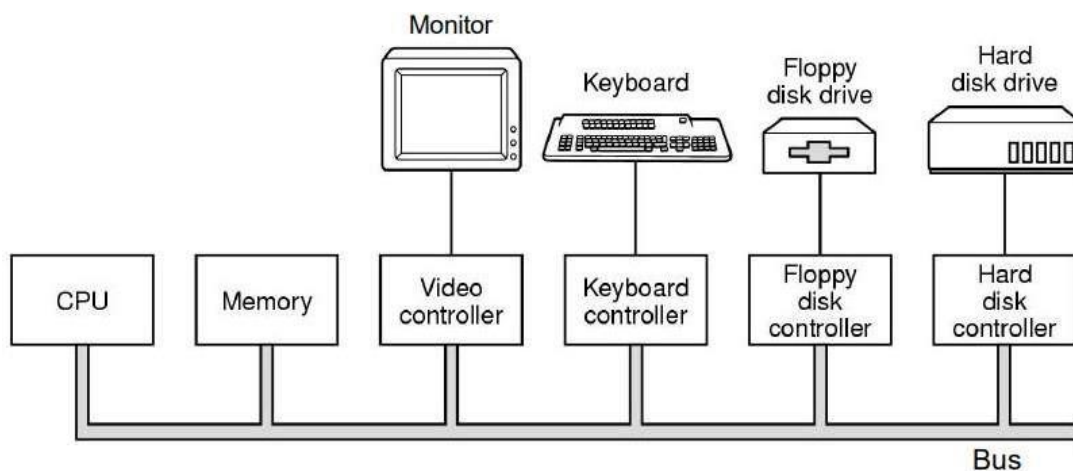
- stores information in a fixed sized block, each one with its own address.
- common block size ranges from 512 bytes to 32,768 bytes
- data transfer takes place in blocks
- block addressable not byte addressable
- e.g. Hard disks, CD-ROMs, USB sticks etc.

2. Character devices

- delivers or accepts a stream of characters, without regard to any block structure
- they are not addressable
- e.g. printers, network interfaces, mice etc.

DEVICE CONTROLLERS

- A device controller is a piece of hardware that receives commands from the system bus, translates them into device actions and reads/writes the data onto the system bus.
- I/O devices typically consists of two components: electronic and mechanical
- The electronic component is called the device controller or the adapter
- A device controller is a part of a computer system that makes sense of the signals going to, and coming from the CPU
- There are many device controllers in a computer system
- Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller
- In personal computer, device controller usually takes the form of a chip on the parent board
- Many controllers can handle two, four or even eight identical devices



Memory-mapped I/O

- Memory-mapped I/O uses the same address bus to address both memory and I/O devices
- The memory and registers of the I/O devices are mapped to (associate with) address values.
- When an address is accessed by the CPU, it may refer to a portion of physical RAM, but it can also refer to memory of the I/O device.
- The CPU instructions used to access the memory can also be used for accessing devices.
- Each I/O device monitors the CPU's address bus and responds to any CPU access of an address assigned to that device, connecting the data bus to the desired device's hardware register.
- To accommodate the I/O devices, areas of the addresses used by the CPU must be reserved for I/O and must not be available for normal physical memory.

DMA – Direct memory access

Direct Memory Access (DMA) is a method that allows an input/output (I/O) device to send or receive data directly to or from the main memory, bypassing the CPU to speed up memory operations. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts. So, a typical computer uses direct memory access (DMA) hardware to reduce this overhead.

Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus. The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.

Advantage

- Free up CPU
- high transfer rates
- fewer CPU cycles for each transfer

Disadvantages

- DMA transfer requires a DMA controller to carry out the operation, hence more expensive system
- synchronization mechanisms must be provided in order to avoid accessing non-updated information from RAM

Use CPU over DMA if

- device speed is fast

- CPU has nothing to do (if cpu is free)

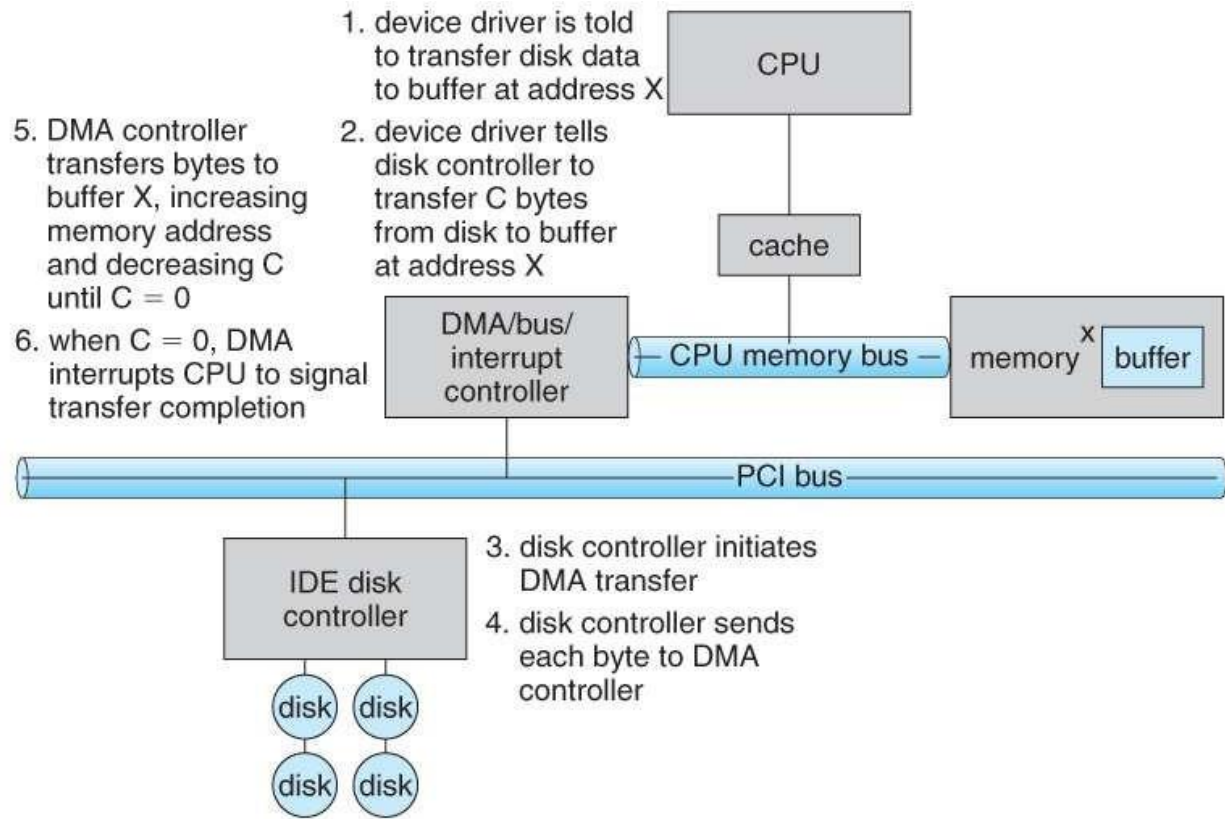


Fig: Block diagram and working of DMA

PRINCIPLES OF I/O SOFTWARE

Goals of I/O software

1. Device independence

- It should be possible to write programs that can access any I/O device without having to specify the device in advance.
- For example, a program that reads a file as input should be able to read a file on a hard disk, a CD-ROM, a DVD, or a USB without having to modify the program for each different device.

2. Uniform naming

- The name of the device should simply be a string or an integer and do not depend on the device in any way.

3. Error Handling

- In general, the error should be handled as close to the hardware as possible
- For example, if controller discovers read error, it should try to correct the error itself. If it cannot, the device driver should handle it.

4. Synchronous (blocking) Vs. Asynchronous (interrupt driven) transfers

- It is up to the OS to make the operation that are interrupt driven look blocking to the user programs.

5. Buffering

- Often, data that comes off a device cannot be stored directly to its final destination due to various reasons.
- buffer is the intermediate destination.

6. Dedicated Vs Shared devices

- Devices that can be used by many users at a time are sharable. E.g. Disk (multiple users can open files from the same disk at the same time)
- Some devices have to be dedicated to a single person. E.g. tapes

Basically, input/output software organized in the following four layers:

- Interrupt handlers
- Device drivers
- Device-independent input/output software
- User-space input/output software

In every input/output software, each of the above given four layer has a well-defined function to perform and a well-defined interface to the adjacent layers.

The figure given below shows all the layers along with hardware of the input/output software system.

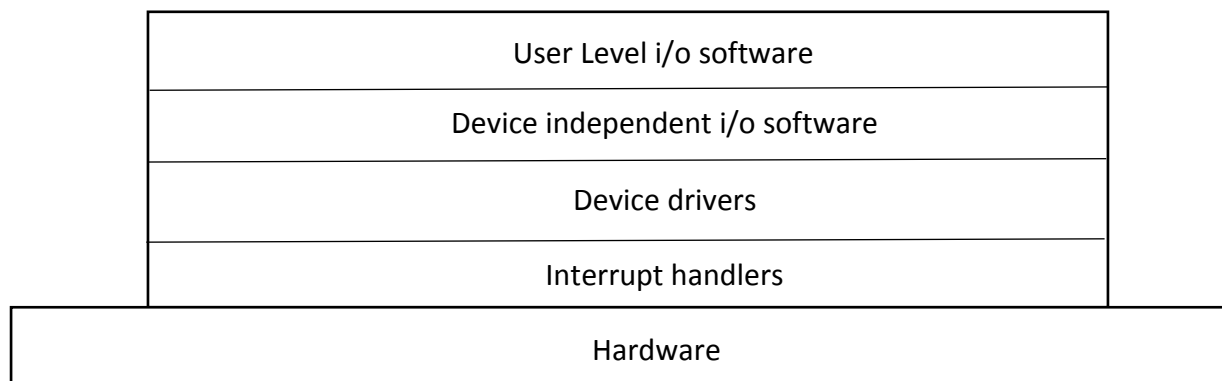


Figure: Layers of i/o software

DEVICE DRIVERS

- Device specific code for controlling the device is called device driver.
- Written by device's manufacturer and delivered along with the device.
- Device driver is the software part that communicates to the device controller, giving it commands and accepting response
- A driver provides the software interface to hardware devices, enabling the OS and other

programs to access hardware functions without having to know the precise details of the hardware being used.

FUNCTIONS OF DEVICE DRIVERS

- i. Accept read/write request from device independent I/O software above it.
- ii. Initialize the device if needed.
- iii. Manage its power requirements and log events
- iv. Check whether the input parameters are valid
- v. Translate parameters from abstract to concrete terms (e.g, linear block number to CHS (Cylinder Head, Sector) for disk)
- vi. Check if the device is currently in use. If it is request will be queued for later processing.

Interrupt handlers

An interrupt handler, also known as an interrupt service routine or ISR, is a piece of software or more specifically a callback function in an operating system or more specifically in a device driver, whose execution is triggered by the reception of an interrupt.

When the interrupt happens, the interrupt procedure does whatever it has to in order to handle the interrupt, updates data structures and wakes up process that was waiting for an interrupt to happen. The interrupt mechanism accepts an address — a number that selects a specific interrupt handling routine/function from a small set. In most architectures, this address is an offset stored in a table called the interrupt vector table. This vector contains the memory addresses of specialized interrupt handlers.

DEVICE INDEPENDENT I/O SOFTWARE

Although some of the I/O software is device specific, other parts of it are device independent. The basic function of device-independent software is to perform the I/O functions that are common to all devices and provide uniform interface to user-level software.

Functions of device independent I/O software

- i. Uniform interfacing for device drivers
- ii. Buffering
- iii. Error Reporting
- iv. Allocating and releasing dedicated devices
- v. Providing a device-independent block size

User-Space I/O Software

These are the libraries which provide richer and simplified interface to access the functionality of the kernel or ultimately interactive with the device drivers. Most of the user-level I/O software

consists of library procedures with some exception like spooling system which is a way of dealing with dedicated I/O devices in a multiprogramming system.

I/O Libraries (e.g., stdio) are in user-space to provide an interface to the OS resident device-independent I/O SW. For example putchar(), getchar(), printf() and scanf() are example of user level I/O library stdio available in C programming.

Polled driven IO and Interrupt driven IO

Most input and output devices are much slower than the CPU—so much slower that it would be a terrible waste of the CPU to make it wait for the input devices. For example, compare the speed you can type with the speed the CPU can execute instructions. Even a very fast typist could probably type no more than 10 characters per second, while a modern CPU can execute more than two billion instructions in that same second!

Polled driven

If the CPU simply waits for the next input character, we call this polled I/O. In this method, the CPU continuously polls the I/O device in a program loop, waiting for the next input. Think of a game where a basketball player is asked to make as many free-throws as possible in one minute, but the clock is down the hall in another room. The player must run down the hall and check if the minute has passed, and if not, go back to the gym and try to make another shot, then run down the hall to check the clock and run back to take another shot. The player spends much of the time simply checking (polling) the clock.

Interrupt driven

An alternative scheme for dealing with I/O is the interrupt-driven method. Here the CPU works on its given tasks continuously. When an input is available, such as when someone types a key on the keyboard, then the CPU is interrupted from its work to take care of the input data. In our example, the basketball player would take shots one after another, while a second person watched the clock down the hall. When the clock reached one minute, the person watching the clock would yell down the hallway for the player to stop. This allows the player to take many more shots, but at the expense of needing someone to watch the clock. Similarly, the CPU can work continuously on a task without checking the input devices, allowing the devices themselves to interrupt it as necessary. This requires some extra "smarts" in the form of electronic circuitry at the I/O devices so that they can interrupt the CPU.

CUI and GUI

GUI and CUI are two types of User Interfaces. GUI stands for Graphical User Interface while CUI stands for Character User Interface. In this article we will be discussing the differences between these two interfaces and which one has advantages over the other.

User Interface: User Interface comprises of everything the user can use to interact with the computer. It is basically the means by which the user and computer system can interact using input and output devices.

GUI: GUI stands for Graphical User Interface. This is a type of user interface where user interacts with the computer using graphics. Graphics include icons, navigation bars, images etc. Mouse can be used while using this interface to interact with the graphics. It is a very user-friendly interface and requires no expertise. Eg: Windows has GUI.

CUI: CUI stands for Character User Interface. This is a type of user interface where user interacts with computer using only keyboard. To perform any action a command is required. CUI is precursor of GUI and was used in most primitive computers. Most modern computers use GUI and not CUI. Eg: MS-DOS has CUI.

Disk

- Disk is an I/O device that is common to every computer.
- Disks comes in many sizes and speeds, and information may be stored optically or magnetically; however, all disks share a number of important features.

Disks are the information storage media

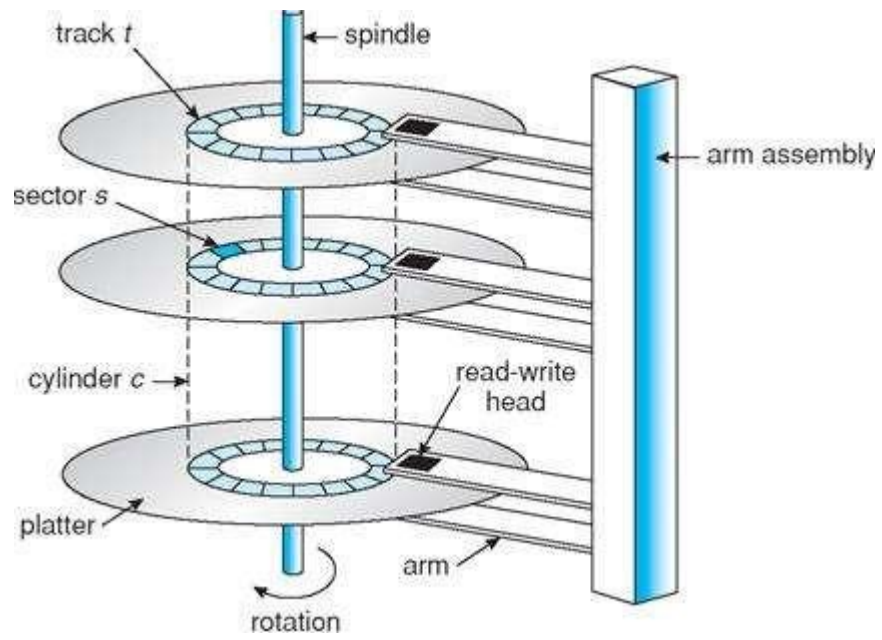
- Advantages of disks over main memory:
 - Storage capacity of disk is greater than main memory
 - Disks are non-volatile (i.e. unlike main memory, the information on disk is not lost even after power off)
- Disks come in variety of types:

-Magnetic Disks : Hard Disks, Floppy disks

-Optical Disks : CD-ROMs, DVDs etc.

A disk consists of several platters

- Each disk platter has a flat circular shape, like a CD
- The two surfaces of a platter are covered with a magnetic material
- A read-write head "flies" just above each surface of every platter
- The heads are attached to a disk arm that moves all the heads as a unit
- The surface of a platter is logically divided into circular tracks, which are subdivided into sectors
- The set of tracks that are at one arm position makes up a cylinder
- There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors



Disk scheduling

- Disk scheduling is done by operating systems to schedule I/O requests arriving for disk.
- Disk scheduling is also known as I/O scheduling.
- OS is responsible to use the hardware efficiently for the disk drive this means fast seek, latency and transmission time.
- For most disks, the seek time dominates the other two times, so reducing the mean seek time can improve system performance substantially.
- Disk I/O time = seek time + rotational delay + transfer time

Seek: Position head over desired cylinder/track. Seek time depends on how fast the hardware moves the arm.

Rotational Delay: time for the sector to rotate underneath the head. Rotational delay depends on how fast the disk spins.

Transfer time: time to move the bytes from disk to memory.

Disk Scheduling Algorithms

1. **FCFS (First Come First Serve)**

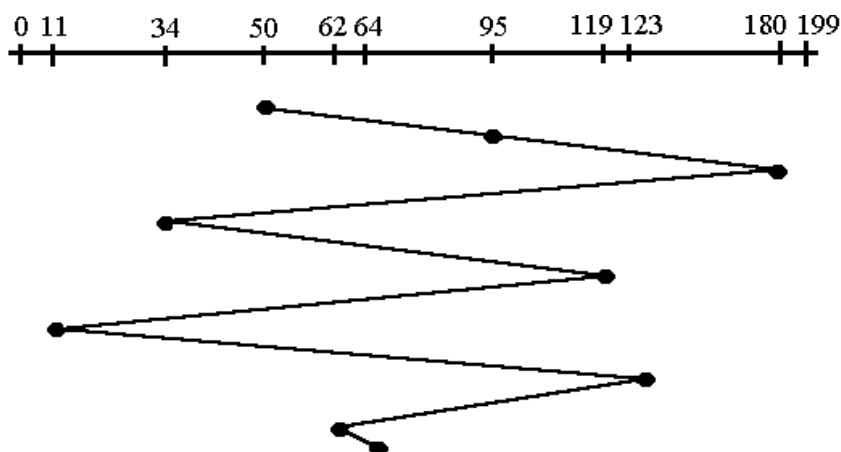
FCFS is the simplest of all the Disk Scheduling Algorithms. In FCFS, the requests are addressed in the order they arrive in the disk queue.

Advantages:

- Every request gets a fair chance i.e no starvation
- No indefinite postponement

Disadvantages:

- Does not try to optimize seek time
- May not provide the best possible service
- **Example:** Given the following queue: 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head initially at the track 50 and total track is 200(0-199). Find total head movement and average head movement.



$$\begin{aligned}\text{Total distance travelled} &= (95-50) + (180-95) + (180-34) + (119-34) + (119-11) + (123-11) + (123-62) + (64-62) \\ &= 640 \text{ tracks}\end{aligned}$$

$$\text{Average distance} = \text{Total distance travelled} / \text{number of track} = 640 / 8 = 80 \text{ tracks}$$

2. SSTF(Shortest Seek Time First)

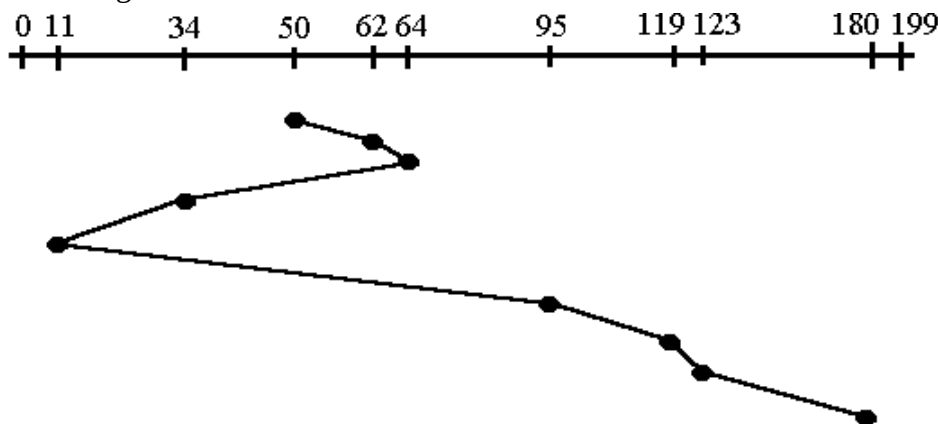
In SSTF (Shortest Seek Time First), requests having shortest seek time are executed first. So, the seek time of every request is calculated in advance in queue and then they are scheduled according to their calculated seek time. As a result, the request near the disk arm will get executed first. SSTF is certainly an improvement over FCFS as it decreases the average response time and increases the throughput of system.

Advantages:

- Average Response Time decreases
- Throughput increases

Disadvantages:

- Overhead to calculate seek time in advance
- Can cause Starvation for a request if it has higher seek time as compared to incoming requests
- High variance of response time as SSTF favors only some requests
- Switching directions causes things slow down
- **Example:** Given the following queue: 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head initially at the track 50 and total track is 200(0-199). Find total head movement and average head movement.



$$\text{Total distance travelled} = (62-50) + (64-62) + (64-34) + (34-11) + (95-11) + (119-95) + (123-119) + (180-123) \\ = 236 \text{ tracks}$$

$$\text{Average distance} = \text{Total distance travelled} / \text{number of track} = 236 / 8 = 29.5 \text{ tracks}$$

3. SCAN

In SCAN algorithm the disk arm moves into a particular direction and services the requests coming in its path and after reaching the end of disk, it reverses its direction and again services the request arriving in its path. So, this algorithm works like an elevator and hence also known as **elevator algorithm**. As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.

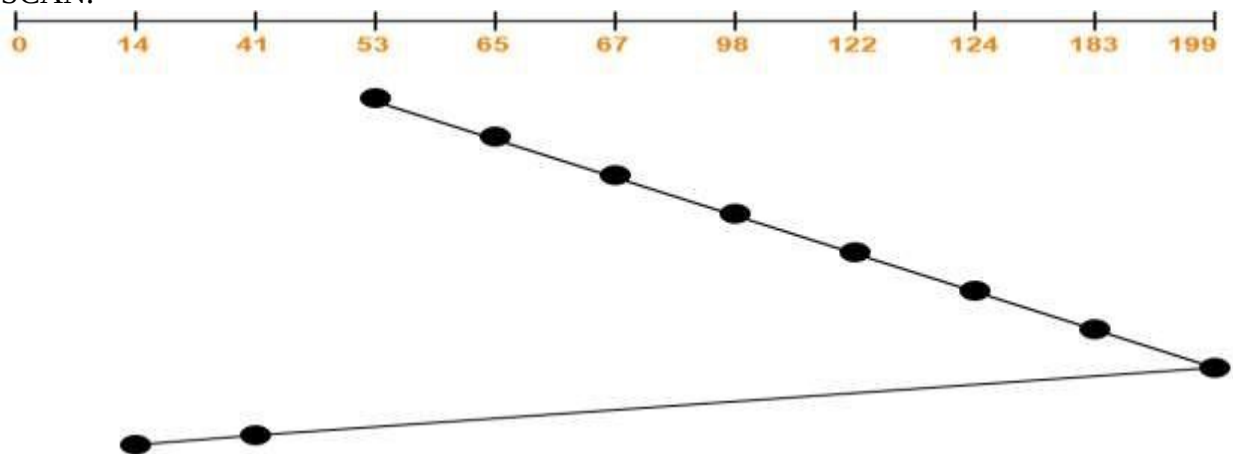
Advantages-

- It is simple, easy to understand and implement.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

Disadvantages-

- It causes long waiting time for the cylinders just visited by the head.
- It causes the head to move till the end of the disk even if there are no requests to be serviced.

Example: Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. Find the cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests using SCAN.



- Total head movements incurred while servicing these requests
- $= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (199 - 41) + (41 - 14)$
- $= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 158 + 27$
- $= 331$ cylinders

4. C-SCAN:

In SCAN algorithm, the disk arm again scans the path that has been scanned, after reversing its direction. So, it may be possible that too many requests are waiting at the other end or there may be zero or few requests pending at the scanned area.

These situations are avoided in CSCAN algorithm in which the disk arm instead of reversing its direction goes to the other end of the disk and starts servicing the requests from there. So, the disk arm moves in a circular fashion and this algorithm is also similar to SCAN algorithm and hence it is known as C-SCAN (Circular SCAN).

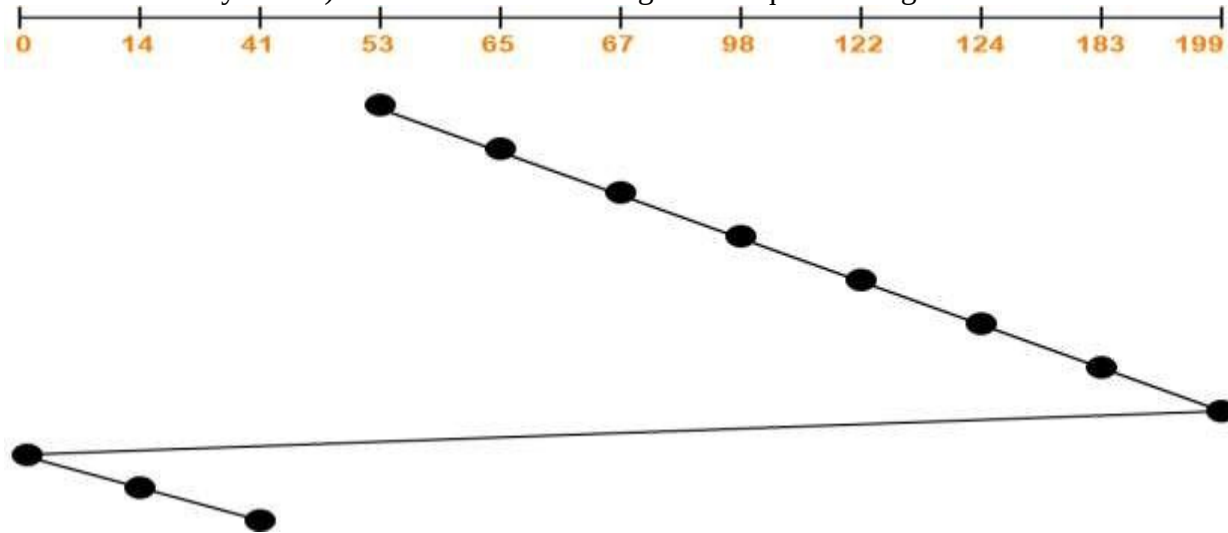
Advantages-

- The waiting time for the cylinders just visited by the head is reduced as compared to the SCAN Algorithm.
- It provides uniform waiting time.
- It provides better response time.

Disadvantages-

- It causes more seek movements as compared to SCAN Algorithm.
- It causes the head to move till the end of the disk even if there are no requests to be serviced.

Example: Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. Find the total head movement (in number of cylinders) incurred while servicing these requests using C-SCAN.



Total head movements incurred while servicing these requests
= $(65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (199 - 183) + (199 - 0) + (14 - 0) + (41 - 14)$
= $12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 27$
= 386 cylinders

5. LOOK:

It is similar to the SCAN disk scheduling algorithm except the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus, it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

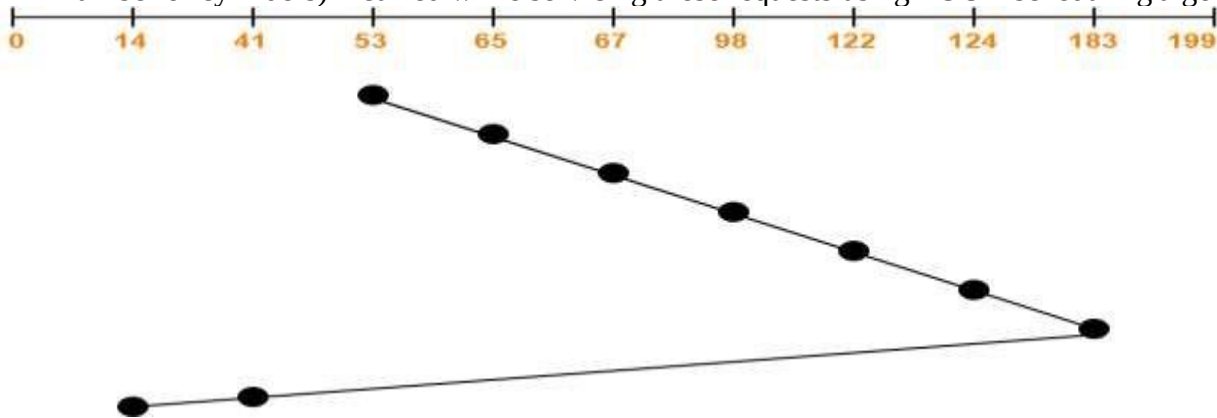
Advantages-

- It does not cause the head to move till the ends of the disk when there are no requests to be serviced.
- It provides better performance as compared to SCAN Algorithm.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

Disadvantages-

- There is an overhead of finding the end requests.
- It causes long waiting time for the cylinders just visited by the head.

Example: Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. Find the total head movement (in number of cylinders) incurred while servicing these requests using LOOK scheduling algorithm.



Total head movements incurred while servicing these requests
 $= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (183 - 41) + (41 - 14)$
 $= 12 + 2 + 31 + 24 + 2 + 59 + 142 + 27$
 $= 299$ cylinders

6. CLOOK

As LOOK is similar to SCAN algorithm, in similar way, CLOOK is similar to CSCAN disk scheduling algorithm. In CLOOK, the disk arm inspite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

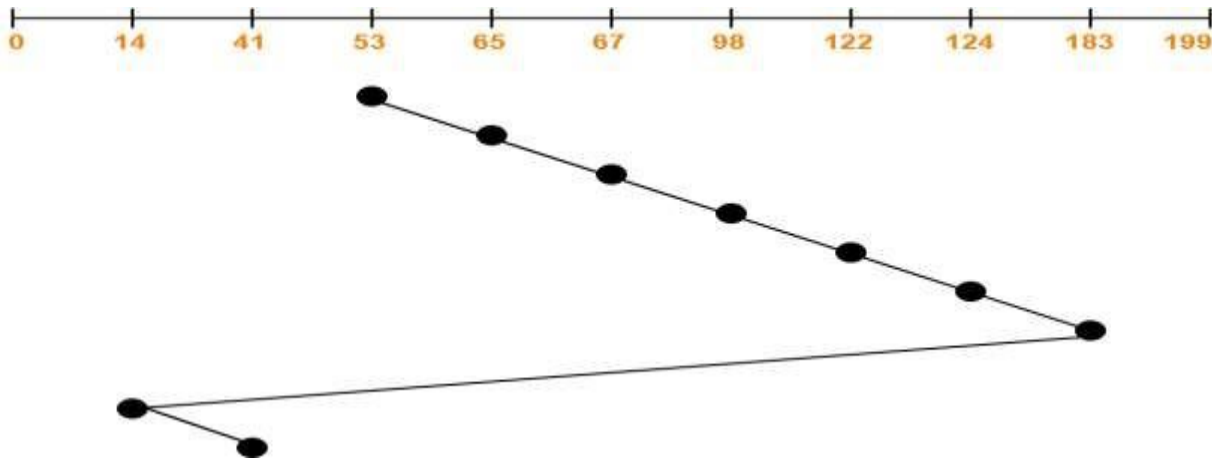
Advantages-

- It does not cause the head to move till the ends of the disk when there are no requests to be serviced.
- It reduces the waiting time for the cylinders just visited by the head.
- It provides better performance as compared to LOOK Algorithm.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

Disadvantages-

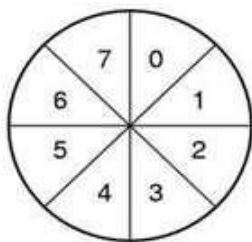
- There is an overhead of finding the end requests.

Example: Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. Find The total head movement (in number of cylinders) incurred while servicing these requests using CLOOK.



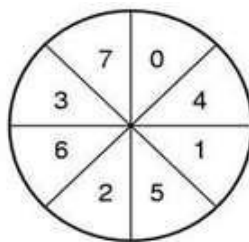
Interleaving

- Consider, a controller with a one-sector buffer that has been given a command to read two consecutive sectors.
- After reading the first sector from the disk and doing the ECC (error correction code) calculation, the data must be transferred to main memory.
- While this transfer is taking place, the next sector will fly by the head.
- When the copy to memory is complete, the controller will have to wait almost an entire rotation time for the second sector to come around again.
- This problem can be eliminated by a technique called interleaving
- The idea is to number the sectors in interleaved fashion rather than sequentially
- Interleaving gives the controller some breathing time between consecutive sectors in order to copy the buffer to main memory
- We may also implement double interleaving if the copying is really slow.



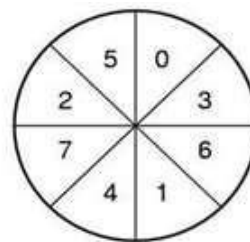
(a)

No interleaving



(b)

Single interleaving



(c)

Double interleaving

RAID

RAID, short for redundant array of independent (originally inexpensive) disks is a disk subsystem (arrangement) that stores your data across multiple disks to either increase the performance,

reliability or provide fault tolerance to the system. RAID can be categorized into Software RAID and Hardware RAID. In software RAID, the memory architecture is managed by the operating system. In case of hardware RAID, there is a dedicated controller and processor present inside the disks that manage the memory. There are various raid levels as discussed below.

Standard RAID levels [RAID 0](#):

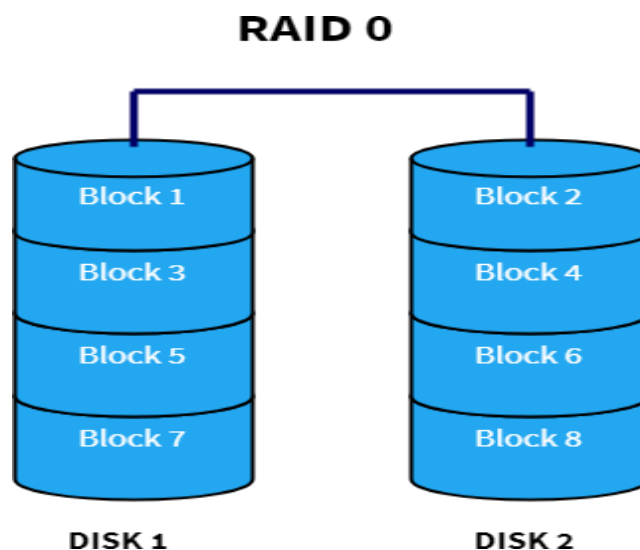
RAID 0 is **based on data striping**. A stream of data is divided into multiple segments or blocks and each of those blocks is stored on different disks. So, when the system wants to read that data, it can do so simultaneously from all the disks and join them together to reconstruct the entire data stream. The benefit of this is that the speed increases drastically for read and write operations.

Advantages:

- Performance boost for read and write operations.
- Space is not wasted as the entire volume of the individual disks are used up to store unique data.

Disadvantages

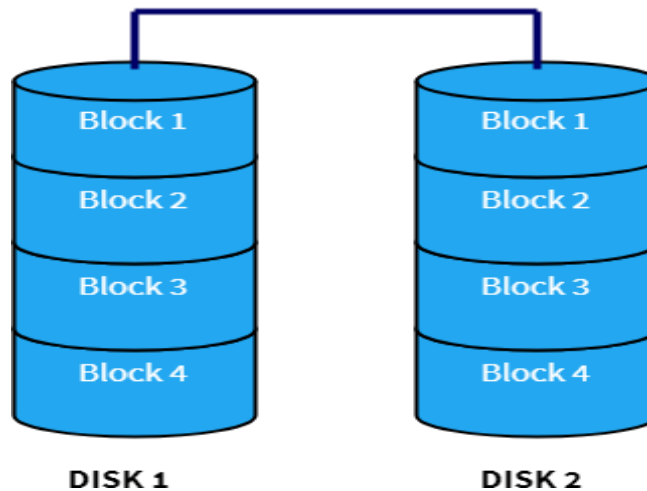
- There is no redundancy/duplication of data. If one of the disks fails, the entire data is lost.



[RAID 1](#):

RAID 1 uses the **concept of data mirroring**. Data is mirrored or cloned to an identical set of disks so that if one of the disks fails, the other one can be used. It also improves read performance since different blocks of data can be accessed from all the disks simultaneously.

RAID 1



Advantages

- Data can be recovered in case of disk failure
- Increased performance for read operation

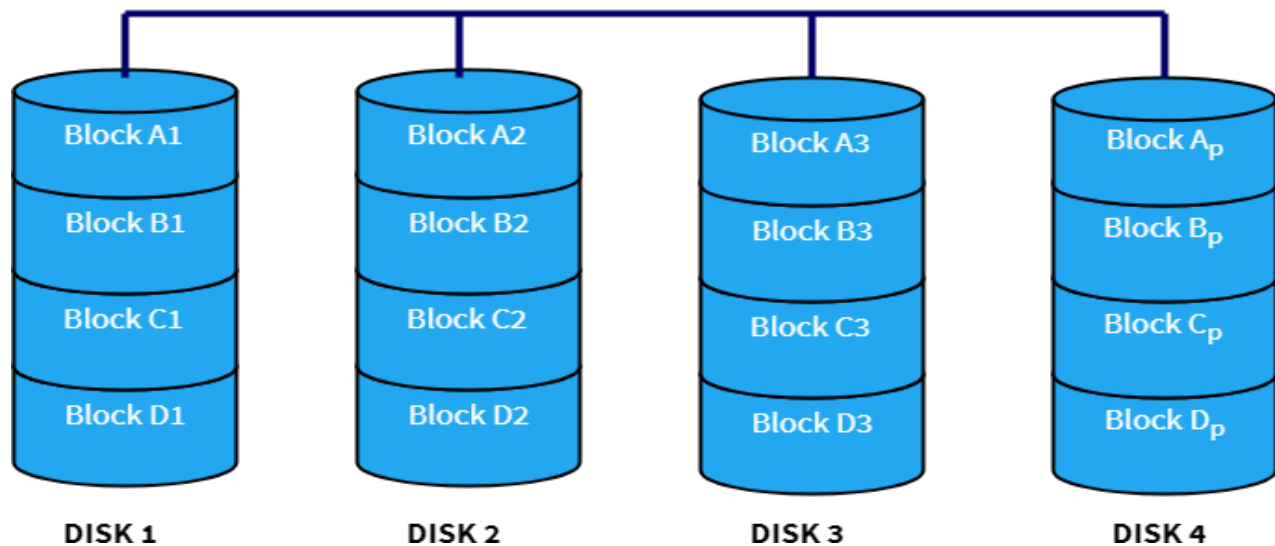
Disadvantages

- Slow write performance
- Space is wasted by duplicating data which increases the cost per unit memory

RAID 4:

RAID 4 stripes the data across multiple disks just like RAID 0. In addition to that, it also stores parity information of all the disks in a separate dedicated disk to achieve redundancy. In the diagram below, Disk 4 serves as the parity disk having parity blocks Ap, Bp, Cp and Dp. So, if one of the disks fails, the data can be reconstructed using the parity information of that disk. **Space is more efficiently used here when compared to RAID 1 since parity information uses way less space than mirroring the disk.**

RAID 4



Advantages

- Efficient data redundancy in terms of cost per unit memory
- Performance boost for read operations due to data stripping

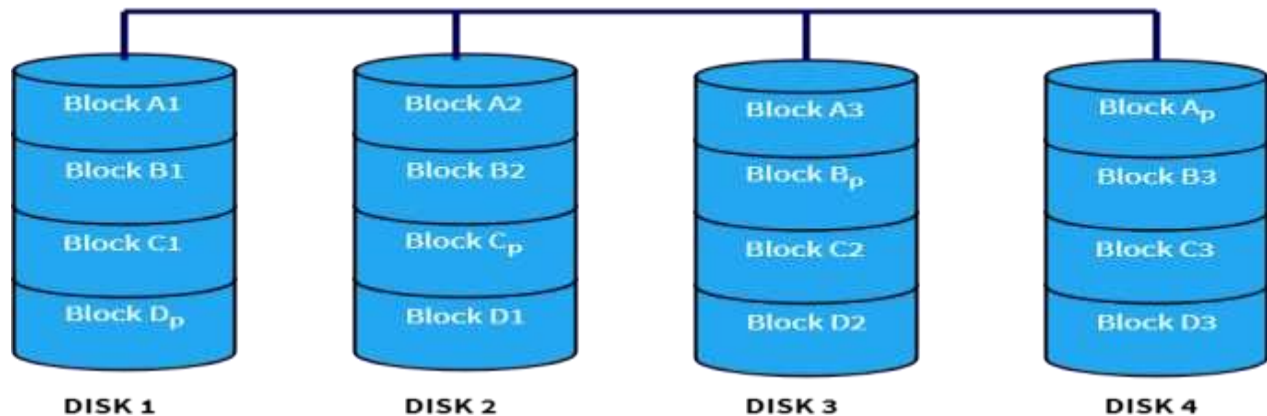
Disadvantages

- Write operation is slow because all the parity information is written on a single disk which is a bottleneck.
- If the dedicated parity disk fails, data redundancy is lost

RAID 5

RAID 5 is very similar to RAID 4, but here the parity information is distributed over all the disks instead of storing them in a dedicated disk. This has two benefits — First, there is no more a bottleneck as the parity stress evens out by using all the disks to store parity information and second, there is no possibility of losing data redundancy since one disk does not store all the parity information.

RAID 5



Advantages

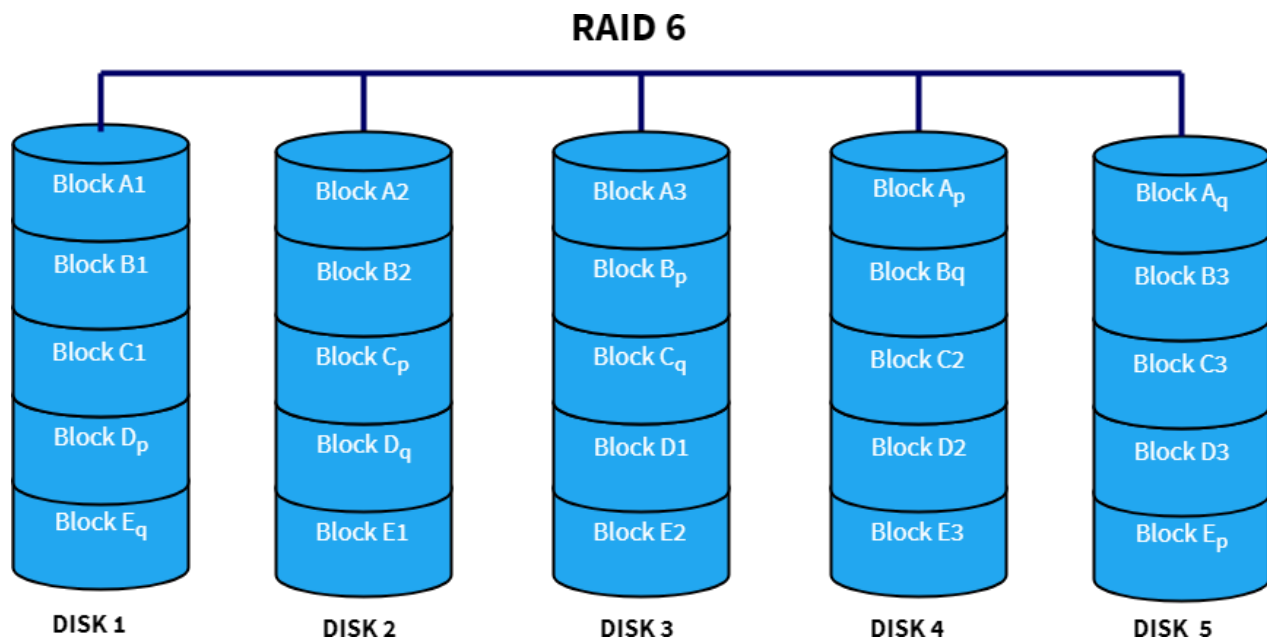
- All the advantages of RAID 4 plus increased write speed and better data redundancy

Disadvantages

- Can only handle up to a single disk failure

RAID 6

RAID 6 uses double parity blocks to achieve better data redundancy than RAID 5. This increases the fault tolerance for up to two drive failures in the array. Each disk has two parity blocks which are stored on different disks across the array. RAID 6 is a very practical infrastructure for maintaining high availability systems.



Advantages

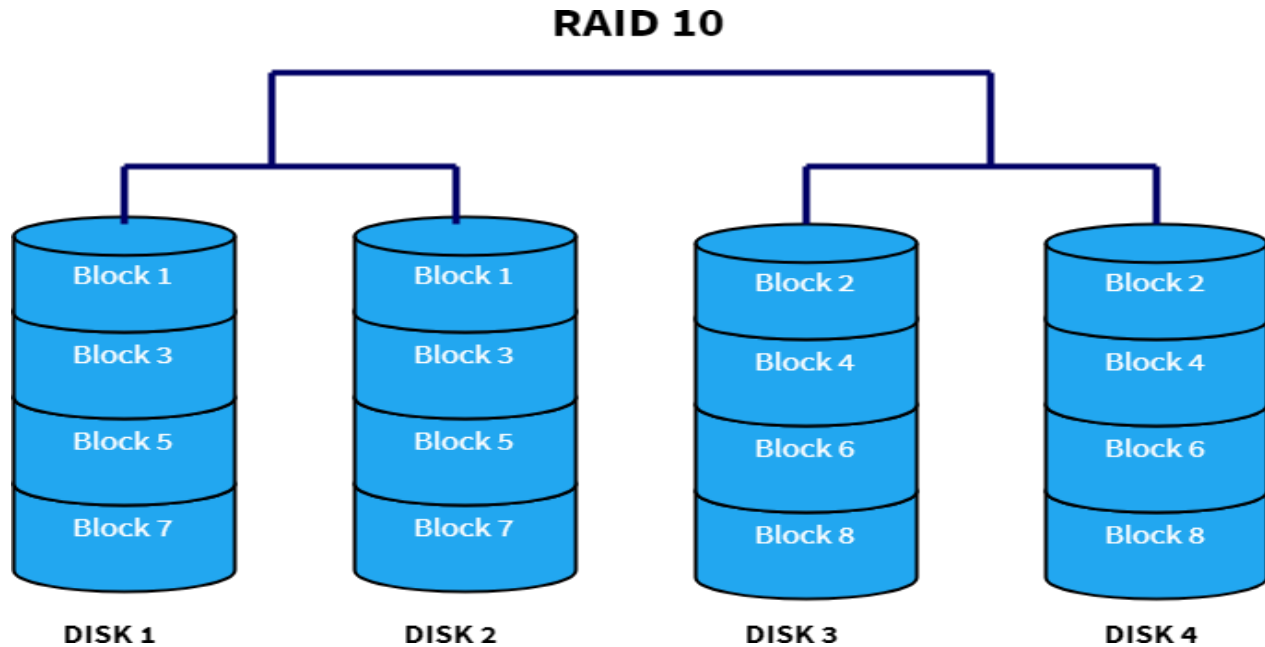
- Better data redundancy. Can handle up to 2 failed drives

Disadvantages

- Large parity overhead

RAID 10 (RAID 1+0)

RAID 10 combines both RAID 1 and RAID 0 by layering them in opposite order. Sometimes, it is **also called as “nested” or “hybrid” RAID**. This is a “best of both worlds approach”, because it has the fast performance of RAID 0 and the redundancy of RAID 1. In this setup, multiple RAID 1 blocks are connected with each other to make it like RAID 0. It is used in cases where huge disk performance (greater than RAID 5 or 6) along with redundancy is required.



Advantages

- Very fast performance
- Redundancy and fault tolerance

Disadvantages

- Cost per unit memory is high since data is mirrored

What about RAID levels 2, 3 and 7?

These levels do exist but are not that common.

RAM Disk

RAM Disk also known as RAM Drive is not a disk at all; It is a program which takes the portion of the system memory (RAM, in the form of memory chips) and uses it as a disk drive. You can create a larger RAM disk, if your computer has more RAM.

A RAM disk can also take the form of a hardware device or a virtual disk. RAM drive makes use of regular RAM instead of accessing the data bus.

As long as the power is on, you can use a RAM disk just like a real disk drive-you can copy files to and from the RAM disk, display the RAM disk's directory or folder on your screen, and run any programs you've stored there.

The significant advantage of a RAM disk is that because it's purely electronic (rather than a physical platter), it's much, much faster than any mechanical disk-hard, floppy, or optical. Typically, if a broad application or program can't fit into regular memory all at once, the computer has to go to the hard disk from time to time and retrieve what it needs to continue working, which slows down the entire process. But if you copy the application to your RAM disk and run it from there, you'll barely notice when the program fetches more instructions from the RAM disk.

RAM disappears when the power goes out, or the system crashes. If you make changes in a document file parked on a RAM disk, you still must be careful to save the file to a real drive. Unless you're willing to go long periods without saving "permanently" and risk of losing a lot of work, putting your documents on a RAM disk will slow you down.

RAM disks have another disadvantage: the memory they use is unavailable for other programs.

Advantages of RAM Disk

The benefits of RAM disk are as follows:

- It makes the performance of I/O process faster and more efficient.
- The users make use of a RAM disk to speed up their applications like games, audio, and video editing, databases, etc.
- The performance of a RAM Disk is higher.
- The main benefit of RAM disk is you can access the RAM disk at maximum bandwidth without any fear of mechanical failure.
- Also, at maximum bandwidth, it does not produce excessive heat, noise or vibration.

Disadvantages of using RAM Disk

The disadvantages of using RAM Disk are as follows:

- Its most significant disadvantage is its cost. As we know that, RAM is very costly as compared to a hard disk.
- If your computer lost power and your system gets crashed, then all of your saved data also get lost.
- RAM disk reserves a good portion of your memory so that you cannot use that memory for any other purpose.

Applications of using a RAM disk

A RAM disk is like a hard drive, but it makes the use of memory. You can use a RAM disk like: you can copy applications and files to the RAM disk, and then you can start those applications or those files from there if the program allows that.

Mostly, RAM disks are suitable for those

- who want to store temporary data.
- You can also set the Windows temp folder location there for instance.

Whenever you restart your computer, this data got removed entirely automatically, and you do not need to do this manually.

Terminals

- Terminals are data input-output device usually made up of a monitor (display), keyboard, mouse, or touch screen.
- A computer terminal is an electronic or electromechanical hardware device that is used for entering data into, and displaying or printing data from, a computer or a computing system.
- It is the point at which a user is connected to and communicates with a computer or a website through a network.

Three basic types of terminals are:

1. **Dumb terminal:** has no built-in data processing capabilities and serves only to send and receive data.
2. **Smart terminal:** has limited data processing capabilities.
3. **Intelligent terminal:** has a substantial data processing capability to inbuilt processor and memory.